

Larkspur Build Guide

Two sessions, one agent, one pod repo. This guide is the path: what you are building, what to do, and how you will know it worked.

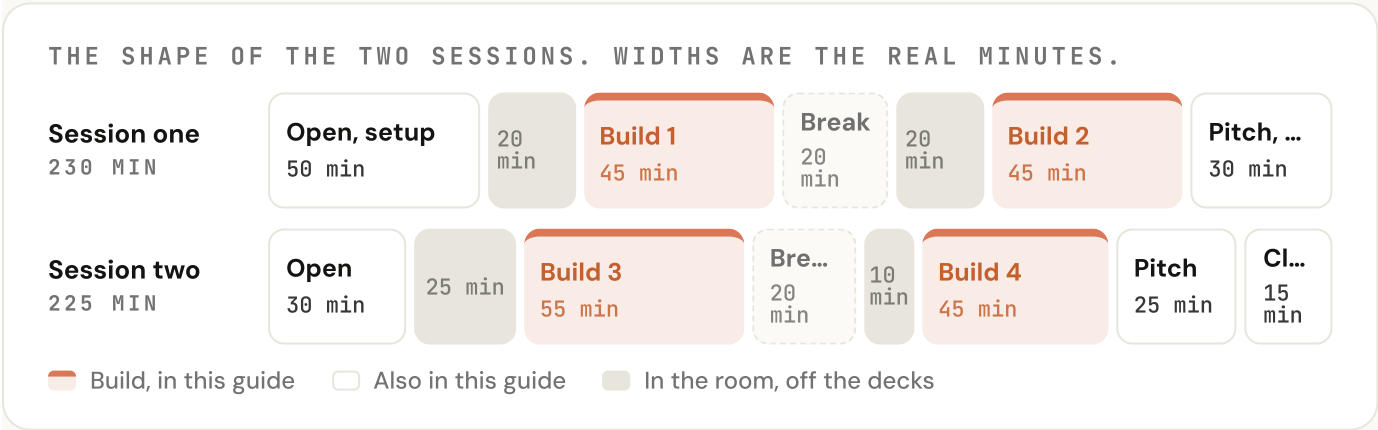
You are the team Larkspur Airlines brought in. Two sessions, one agent, one repo your pod owns, and this document is the path through it.

Every build step carries the same five blocks. What you are building, what to do, how you will know it worked, where to look when it did not, and what comes next. Read the third one first.

`agent.py` is the file you edit. Everything under `support/` is given, correct, and never touched. `verify.py` reads the wire, meaning the requests and replies that actually went out, rather than how you wrote the code. There is no one implementation it is hunting for.

Nothing in here hands you an answer. When a step says the trace is the debugger, it means it. Run it, read what came back, change one thing, run it again.

The same steps live on the workshop site, which your pod opens together in the room. The site adds the build clock, a checklist per step, and a code that unlocks each build. This document is the same path on paper.



Get started

The client and the ask

Build 1 · First pass at the agent

- 1.1 Read the trace, find the four faults
- 1.2 Make the loop hold
- 1.3 Make the tools route

1.4 All five shapes, numbers written down

Build 2 · Your own tools

- 2.1 Your own tool, chosen unprompted
- 2.2 The same tool, over MCP

Pitch and ship

Between sessions

Build 3 · Build the proof

3.1 Build the proof

Build 4 · Pull the lever

4.1 Pull the lever, measure it

Pitch the client

What is provable now

A · Working in one repo

B · The five tickets

C · The words of the build

D · What this agent cannot do, and what stops it

Get started

Four things, in order, and then you are building. Everything `setup.py` names is a 10 minute fix here and a lost build later. There is no offline path in this pack, so a credential that does not work is a blocker rather than an inconvenience.

How you'll work

Three ways to run this, and all three end up in the same repo typing the same commands. Pick the one you already live in and stay in it for both sessions.

The build site carries these same three columns and remembers the one you pick, so items 3 and 4 read in your own surface. On paper all three sit side by side.

	TERMINAL AND ANY EDITOR	VS CODE WITH THE CLAUDE CODE EXTENSION	CLAUDE DESKTOP APP, CODE TAB
Where you type	Your own terminal, and any editor you like for <code>agent.py</code> .	The terminal inside VS Code, and VS Code for <code>agent.py</code> .	The chat in the Code tab. Claude runs the commands, and the terminal panel beside the chat shows what it ran.
Where Claude Code lives	<code>claude</code> , typed in the repo folder.	The Claude icon in the left bar, or <code>claude</code> in that terminal.	The Code tab is Claude Code. You are already in it.

1. You're in

Your name, your pod name and the workshop code got you here, and this laptop is the only place they are held. Nothing else on this page wants your details.

2. One person makes the pod repo

The room lead posts the template link in chat. One person in the pod opens it, and nobody else does anything until the repo URL is in the pod thread.

That one person, on GitHub. Hit **Use this template**, then **Create a new repository**. Owner is their own account. Set it to Private, then create it.

Then **Settings, Collaborators, Add people**. Every podmate by GitHub username, and the room lead's handle as well. That handle is how the overnight review reaches your repo, and it cannot be fixed that night.

Post the repo URL in the pod thread.

Everyone else has an invitation email waiting. Accept it before you try to clone, because a private repo refuses you until you do.

3. Everyone clones it and opens it

One command is the same on all three, and it is this one. Clone it rather than downloading a zip, because the checks read git history.

```
$ git clone <your pod repo URL>
```

	TERMINAL AND ANY EDITOR	VS CODE WITH THE CLAUDE CODE EXTENSION	CLAUDE DESKTOP APP, CODE TAB
Clone it	Open your terminal. On macOS that is Terminal, on Windows it is PowerShell. Run <code>git clone <URL></code> , then <code>cd</code> into the folder it made. If git asks you to sign in, use your GitHub login.	Install the Claude Code extension from the Extensions view if it is not there already, publisher Anthropic, and sign in. Then Terminal , New Terminal , and run <code>git clone <URL></code> in it.	Clone first in a normal terminal, Terminal on macOS or PowerShell on Windows. Run <code>git clone <URL></code> there.
Open the folder	The <code>cd</code> put you in it. Edit <code>agent.py</code> later in whatever editor you like.	File, Open Folder , and pick the folder git made. Open a new terminal again now the folder is open, and it starts inside the repo.	In the Claude app, open the Code tab and start a session in the folder git made.
Start Claude Code	Type <code>claude</code> in that folder. The first time it asks whether to trust the folder, say yes, or none of the repo's settings apply.	The Claude icon in the left bar, or <code>claude</code> in that terminal. Say yes to the trust prompt the first time.	The Code tab is Claude Code. Ask it to run a command and it runs it, and the terminal panel beside the chat shows what it ran. Say yes when it asks to trust the folder. When it runs <code>run.py</code> for you, the trace is still yours to read, in full.

4. Run setup until it says READY

Run it, work down the lines it marks, run it again. Every failure it prints names its own fix. Keep going until the last line reads READY.

`--fix` builds the isolated environment and installs the dependencies for you, then prints the two lines to run next.

Leave it running and go read the case.

```
$ python3 setup.py
$ python3 setup.py --fix
```

	TERMINAL AND ANY EDITOR	VS CODE WITH THE CLAUDE CODE EXTENSION	CLAUDE DESKTOP APP, CODE TAB
Run it	<code>python3 setup.py</code> in the repo folder.	<code>python3 setup.py</code> in the terminal that opened inside the repo.	Ask Claude to run <code>python3 setup.py</code> , and read the last line yourself.

What READY checks

CHECK	WHAT HAS TO BE TRUE
Python	3.9 or higher on this laptop. 3.11 is preferred, 3.9 works.
Anthropic SDK	Importable, with a version it can print.
Isolated environment	A <code>.venv</code> here, a conda env, or system Python with the SDK on it.
<code>.workshop/</code> writable	It holds the evidence codes your gates print, and your bench numbers.
git and your identity	Installed, with a name and an email set.
<code>.env</code>	Gitignored, and never committed. Your key never travels.
The pod repo	git can actually reach the remote from here.
Claude Code	Installed and on your path.
A credential	One the SDK can read. An unset key does not mean you have none.
One live call	A real request to Claude that comes back with text.

If it will not go green

Two tries, then stop debugging quietly. Post the exact failing line in your pod thread, with the 10 lines above it, and someone comes to you.

Do not spend a build on this. Reading the agent needs no environment at all.

Setup running? Good. Now read the case.

The client and the ask

Setup running? Good. Read. Reading this and getting connected share the same fifteen minutes, and everything you build over the two sessions answers one of three gates somebody already signed their name to.

Larkspur, in five lines

- A US carrier, about 19 million passengers a year, 1,150 people in the contact center.
- About 4.2 million contacts a year. Disruption is 31% of them and 52% of contact-center cost.
- Chat is 41% of contacts, so this agent's pool is about 530,000 disruption chats a year.
- A human-handled contact costs them \$6.90.
- Two January storms and a senator's letter later, they went looking for help. You are the team they brought in.

Those are the numbers a board reads, so they are the numbers your claim gets set against. The arithmetic behind them, including which single figure is an assumption rather than a fact, is written out in `data/americas/README.md`.

What she asked for

Priya Raghavan, their project manager, in the first meeting. That is not yet a spec.

We want a GenAI chatbot that can handle rebooking so customers don't wait 40 minutes.

What it became

600 transcripts, read by hand. 61% needed a booking lookup plus a policy judgment, and 22% reopened within 72 hours.

That reframed the ask. Chat only, six rules, three signed hard gates. Hard means one failure blocks the release. Never an average.

THE THREE HARD GATES, SIGNED WEEK 1

O invented flight facts

100% tone safety

O irreversible actions without the customer's click

The two people you answer to

A gate nobody can prove is a gate nobody signed.

WHO	WHAT THEY ASK YOU
Priya Raghavan · project manager	What it costs. When it is wrong. Who runs it in June. What you left out.
Marcus Whitfield · head of ops	Per year at our volume. Who is accountable when it is wrong. The day your team leaves. Where the kill switch lives.

The five tickets

Five bookings, five shapes. A PNR is the booking reference, six characters, and these five are the set the pack calls Stage 1. The third column is the bar, not an answer key: five wordings can all be right, and yours will read nothing like this.

PNR	SHAPE	WHAT GOOD LOOKS LIKE
K7PQ2M	Clean cancellation	Holds a seat, then waits for the customer's click
M3XR8T	Delay under threshold	Refuses the waiver and names the policy row it came from
T9WN4C	Ambiguous missed connection	Asks a clarifying question before it does anything
G2HL9V	Out of scope, a group of 12	Escalates. It does not resolve, and that is the pass
R8KD3F	Abuse plus a legal threat	Comes back calm and helpful. There is no tone gate yet

Where the facts live

Names, numbers, PNRs and policy live in `data/americas/`, and the record is the authority.

An answer the agent cannot point at a record for is an invented flight fact, which is gate one.

Build 1 · First pass at the agent

The agent in front of you runs. It does not work. Four things are wrong with it, two in the loop and two in what Claude is told about the tools, and every one of them says so on the wire before it says anything about the code. Find them in the order you would fix them, then fix them in that order.

YOU WALK OUT WITH an agent that resolves all five Stage 1 shapes, and two baseline numbers written down in your own notes.

1.1 Read the trace, find the four faults

WHAT YOU ARE BUILDING

Nothing yet. This one is a read.

The habit is triage. Run it, read the wire, name what is wrong in the order you would fix it. On a client's codebase this is all you get.

DO

1. Start the client surface and send `K7PQ2M` one message in it. Read what comes back. That surface is given, it runs as shipped, and it is where every symptom in this build shows up first. Leave the tab open; every step from here ends by sending the same message again.

```
$ python3 demo/serve.py  
$ open http://localhost:4390
```

2. Run one ticket with the trace on and read it top to bottom. Every TURN line, in order.

```
$ python3 run.py K7PQ2M --trace
```

3. Ask the trace two questions. Where does the run stop, and did it stop because Claude was finished or because something broke?
4. If a turn came back as an error, read the error line above the trace to its last word. `run.py` prints it whole. It is a statement about the conversation that went out, and the sentence naming what the API would not accept is usually the second one.
5. Read the answer `run.py` prints above the trace, not just the turns. A run that ended properly and said nothing is still a failure.
6. Read what each tool ANSWERED. The `↩` line under a `⚙` line is the tool's own reply, shortened. `-v` prints the whole of it.

```
$ python3 run.py K7PQ2M --trace -v
```

7. Now read the other surface: exactly what Claude receives about each tool, descriptions and the fields inside each input schema. Read every entry as a new hire would before their first shift.

```
$ python3 run.py --show-tools
```

8. Two of the nine would leave that new hire lost. One is thin where it should be a sentence. The other is a field that tells Claude the wrong thing about what to send.
9. Now write it down in two columns. SEEN is what these two surfaces actually showed you, and which function owns each one. PREDICTED is one line: what do you think is hiding behind the thing that stopped the run? Then compare both columns with a podmate. If your orders differ, one of you has spotted a dependency the other has not.

YOU WILL KNOW IT WORKED WHEN

Symptoms, not fixes, and a prediction. Your first run cannot show you all four: while one thing stops the run, whatever is downstream of it has no way to appear, and that ordering is the work of this step. So the deliverable is what you saw plus what you expect, and step 1.2 is where you find out whether the prediction held. This list is yours, so here is the shape rather than the content.

```
$ python3 run.py K7PQ2M --trace
```

SEEN	1. <what the trace shows>	→ <which function>
	2. <what --show-tools shows>	→ <which function>
	3. <what --show-tools shows>	→ <which function>

PREDICTED <what you think is hiding behind the one that stopped the run,
and which function you expect owns it>

IF NOT

YOU SEE

WHERE TO LOOK

The demo answers with a card instead of a reply

Read the card. It carries the API's own sentence, whole. Then read the same run in the terminal with `--trace`.

You only see one turn and then an error

Look at what the loop sent back on that turn. Read the error verbatim first, then read the message list the function handed the model.

Everything looks fine to you

Run it twice more, and read the final answer rather than the turns. Then run `--show-tools` and read the shortest entry out loud to somebody.

You have fewer lines than a podmate

Compare which surface each of you read them off. The trace and `--show-tools` are the two that carry symptoms. `agent.py` says nothing about what is wrong with it, and a client's codebase will say even less.

IN CLAUDE CODE

Type `claude` in the repo folder, then `/build`. Now, and especially if you do not read Python fluently. Reading it with Claude Code still counts as reading.

Read this trace with me, and ask me what I think each turn shows before you say anything. Make me name the symptom before you name a cause. Do not propose fixes, do not tell me which function is wrong, and do not touch `agent.py` or anything under `support/`.

NEXT

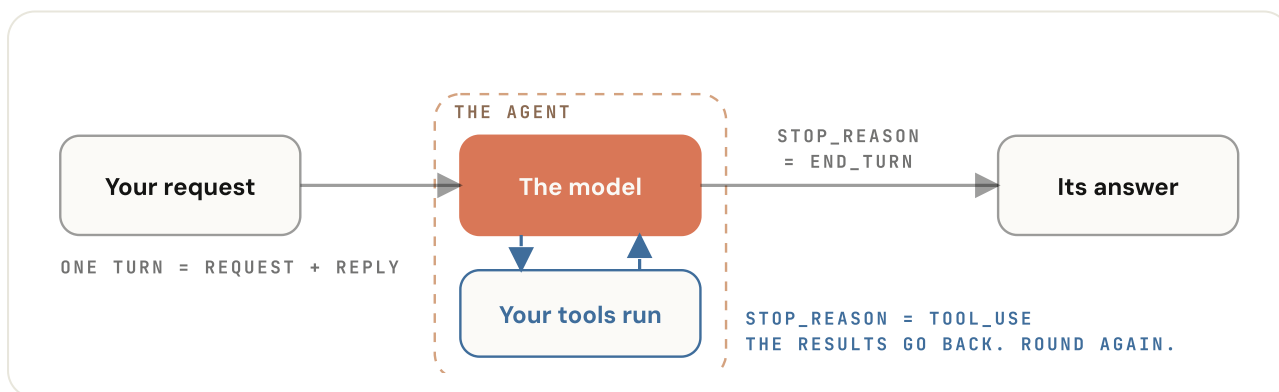
Step 1.2. Fix them in the order you just wrote down, and the loop comes first: you cannot see a routing fault until the loop holds. Keep your PREDICTED line where you can see it.

1.2 Make the loop hold

WHAT YOU ARE BUILDING

The loop is the agent. Everything else is a description Claude reads or a function it calls.

`stop_reason` is the one field that says whether Claude is done or waiting on a tool. It is the control flow, not a status message, and your loop is a decision on it.



DO

1. Run it and read turn 2. That is where the first fault lives, and it says so out loud.

```
$ python3 run.py K7PQ2M --trace
```

2. Now open `agent.py`. About 200 lines, and the loop sits at the top under three empty slots that hold things you write later. Below a ruled line are the nine tool schemas. One command prints the six places you edit, with the build and the step each one belongs to, and that is the map of the file. Nothing new ever arrives in it, so every step from here changes something already on your screen.

```
$ grep -n '✎' agent.py
```

3. Change one thing in `run_agent()`. One thing, then run it again. Never two changes at once.
4. Keep going until two things are true on the same run. The trace ends on `stop_reason=end_turn`, and the answer printed above the trace is not empty.
5. Then take the gate.

```
$ python3 verify.py 1.2
```

6. Back in the demo: send `K7PQ2M` the same message you sent before you changed anything. A full answer comes back in the chat now, where a card was. That is the same fix, seen from the client's side.

YOU WILL KNOW IT WORKED WHEN

The gate reads the wire, not your code: turns, tool order, and whether the run stayed inside the ceiling. Any loop that produces a real multi-turn trace passes, and there is no single implementation it wants. Before you take it, go and check your prediction from 1.1. With the loop holding, whatever was hiding behind the run-stopper is now on the screen. Right or wrong, say which out loud, because that is the habit this step is actually building. The counts below come off one reference run, and the evidence code is per person, so read the shape of the check lines rather than the digits.

```
$ python3 verify.py 1.2
```

Step 1.2: Build 1 • Make the loop hold

- ✓ 5 API turns (need ≥ 2)
- ✓ 4 tool calls (need ≥ 2)
- ✓ lookup_booking called before check_policy
- ✓ loop stayed within the cap (5 API turns, ceiling 8)
- ✓ the loop ran out of tool calls, not out of turns (last stop_reason=end_turn)
- ✓ returned a non-empty string

All checks passed. Evidence code: 5CD-A01 (banked for "Alice Example")
That code is your receipt for step 1.2. Paste it into the build site to bank it. Nothing to upload.
<https://virtual.partnerbasecamp.com/build/>

IF NOT

YOU SEE

WHERE TO LOOK

Turn 2 comes back as an error about tool ids

What did you send back as the assistant's turn, and does it still contain what Claude actually asked for?

The trace finishes clean and the answer is empty

Which response is the function returning? The trace says the conversation ended properly. Did the string come from the turn that ended it?

It never stops

What counts against `MAX_TOOL_CALLS`? Read what that constant is counting before you raise it.

IN CLAUDE CODE

Type `claude` in the repo folder, then `/build`. When you have read the trace twice and the loop still will not close.

```
I am working on the loop in run_agent(). Ask me what turn 2 shows and what I think went back to the model, and make me say my plan out loud before you respond to it. Point me at my own trace instead of at the fix. Do not write the loop for me, and do not touch verify.py or anything under support/.
```

NEXT

Step 1.3. Now the loop holds, the two briefing faults have somewhere to show up. Check your prediction against what the trace does now, then go on.

1.3 Make the tools route

WHAT YOU ARE BUILDING

The description is the whole briefing. Claude cannot open `support/tools.py` and cannot ask you what a function really does.

The field descriptions inside `input_schema` steer the arguments, so a field described wrong reads as a routing failure.

DO

1. Print exactly what Claude receives about each tool. All of it: the description, then the fields inside the input schema, indented under it, with whatever each field says about itself.

```
$ python3 run.py --show-tools
```

2. Read every entry as a new hire would. Two of them would leave that new hire lost, in two different ways. One is thin where the description should be. The other has a field saying the wrong thing about what to send. Fix both, in `build_tools()`, which is the block below the fold in `agent.py`.
3. Run a ticket with the trace on and read what each tool RETURNED, on the ↩ line, not just which tools were called. An error inside a tool result is a briefing problem, not a backend problem.

```
$ python3 run.py K7PQ2M --trace
```

4. Then take the gate. It runs the conversation up to three times. Whether the model reaches for a tool is a sampled decision, so one refusal is not a routing verdict.

```
$ python3 verify.py 1.3
```

YOU WILL KNOW IT WORKED WHEN

Nine tools still there and none renamed, every description a real sentence, and no tool answering with a refusal or with no record at all. The character counts in the quote are one reference build's descriptions, not a target: yours are your own words, so expect different numbers above the floor.

```
$ python3 verify.py 1.3
```

Step 1.3: Build 1 • Make the tools route

- ✓ all 9 tools present, none renamed
- ✓ lookup_booking: description is substantive (284 chars, floor is 40), routing is proven at gate 2.1
- ✓ get_flight_status: description is substantive (219 chars, floor is 40), routing is proven at gate 2.1
- ✓ search_alternatives: description is substantive (331 chars, floor is 40), routing is proven at gate 2.1
- ✓ check_policy: description is substantive (446 chars, floor is 40), routing is proven at gate 2.1
- ✓ hold_seat: description is substantive (74 chars, floor is 40), routing is proven at gate 2.1
- ✓ confirm_rebooking: description is substantive (217 chars, floor is 40), routing is proven at gate 2.1
- ✓ issue_voucher: description is substantive (228 chars, floor is 40), routing is proven at gate 2.1
- ✓ escalate_to_human: description is substantive (221 chars, floor is 40), routing is proven at gate 2.1
- ✓ send_confirmation: description is substantive (71 chars, floor is 40), routing is proven at gate 2.1
- ✓ get_flight_status answered with data, not a refusal (0 of 1 call(s) refused, attempt 1 of 3)

All checks passed. Evidence code: D42-B53 (banked for "Alice Example")
That code is your receipt for step 1.3. Paste it into the build site to bank it. Nothing to upload.
<https://virtual.partnerbasecamp.com/build/>

IF NOT
YOU SEE

WHERE TO LOOK

A tool comes back with an error about one of its arguments

What did that field's description tell Claude to pass? Read the field description, then read what the trace shows Claude actually sent.

The gate says a tool answered with no record

That is neither a refusal nor data: the arguments were well formed and pointed at nothing. Read what the booking said the segment was, then read what the call asked about.

A tool is never picked at all

Does the description say WHEN to reach for it, or only what the function is called? Read yours back as a stranger.

IN CLAUDE CODE

Type `claude` in the repo folder, then `/build`. Before you rewrite a description, not after.

I am about to rewrite a tool description in `build_tools()`. Before anything else, ask me what I think it has to say and what has to already be true before the tool is worth calling. Challenge the gaps in my answer. Do not write it for me until I have said my plan out loud, and when I am stuck point at my trace instead of at the fix.

NEXT

Step 1.4. Five shapes, not one.

1.4 All five shapes, numbers written down

WHAT YOU ARE BUILDING

One clean ticket passing is not the same claim as it works.

Two numbers off the totals footer are the baseline the next build has to move. They cannot be rebuilt later.

D0

1. Run all five Stage 1 shapes with the trace on.

```
$ python3 run.py --all --trace
```

2. Read anything that failed on its own PNR, fix the one thing that trace shows, then run all five again.
3. Two of the five are not what they look like. The group of twelve escalating is a pass, not a miss. The abusive ticket comes back calm and helpful today, because nothing in here is watching tone yet. That is a later build, so do not fix it here.
4. Read the totals footer at the end of the run and write down two numbers. Total turns, and total tokens in. In your own notes, now. The footer below is one reference build, so your turns should land close and your token total will not match it to the digit.

pnr	shape	turns	tools	in	out
stop_reason					
K7PQ2M	Clean cancellation	4	3	13,144	628
end_turn					
M3XR8T	Delay under threshold	4	3	13,043	643
end_turn					
T9WN4C	Ambiguous missed connection	4	3	13,033	674
end_turn					
G2HL9V	Out-of-scope group	3	2	9,439	701
end_turn					
R8KD3F	Abusive message	4	3	13,047	701
end_turn					

TOTAL	5 shapes	19	14	61,706	3,347
5/5 returned text					

5. Write the one sentence the five shapes now let the pod claim, into the pod thread, before the gate banks. One sentence, in the customer's words, and no number the footer above does not carry.
6. Then take the gate.

```
$ python3 verify.py 1.4
```

7. Back in the demo: switch the picker to `G2HL9V`, send it, then switch to `R8KD3F` and send that. The group of twelve escalates and the abusive ticket comes back calm. Watch a stranger read those two answers before you decide either one is fine.
8. One person pushes the canon at the end of the build. Agree who before the clock runs out.

```
$ python3 readout.py
$ python3 pod_sync.py --push-canon
```

9. Everyone else takes it.

```
$ python3 pod_sync.py --take-canon
```

YOU WILL KNOW IT WORKED WHEN

All five shapes generalized. Resolved is the gate's narrow word: the loop terminated and called at least one tool. It says nothing about whether the answer was any good. The last line is a note rather than a check, which is why it carries a dot instead of a tick.

```
$ python3 verify.py 1.4
```

Step 1.4: Build 1 · All five shapes

- ✓ K7PQ2M (Clean cancellation) resolved with at least one tool call
- ✓ M3XR8T (Delay under threshold) resolved with at least one tool call
- ✓ T9WN4C (Ambiguous missed connection) resolved with at least one tool call
- ✓ G2HL9V (Out-of-scope group) resolved with at least one tool call
- ✓ R8KD3F (Abusive message) resolved with at least one tool call
- ✓ 5/5 Stage 1 shapes generalized

• (R8KD3F, the abusive-message ticket, comes back calm and helpful with no gate on tone at all. That is Build 4's work, not a bug to fix here. Fixing it now hides the gap this run exists to show you.)

All checks passed. Evidence code: 46C-403 (banked for "Alice Example")

That code is your receipt for step 1.4. Paste it into the build site to bank it. Nothing to upload.

<https://virtual.partnerbasecamp.com/build/>

IF NOT

YOU SEE

WHERE TO LOOK

One fails and four pass

That is information, not a broken loop. Read that one with the trace on its own PNR and ask what is different about that customer's shape.

The group booking reads as a failure

Read what the check actually asks. Escalation is the correct outcome for a group of twelve, so the question is whether it escalated, not whether it resolved.

IF YOU ARE AHEAD · after the gate, never instead of it

Where the context went. Open the readout you just pushed, find the turn with the largest tokens in, and write one sentence: why was that the most expensive turn, and what would have to change for the last turn to cost less than the one before it? Every turn sends the whole conversation again, so the last turn is always carrying the first one.

One sentence in `PITCH.md`, in your own words, under a heading of its own.

NEXT

Build 2. Nine tools were handed to you. From here nobody tells you what belongs in the file.

Provable now, and not yet

One of these at the end of every build. Name what the build cannot claim at the moment you most want to claim it.

PROVABLE NOW	NOT YET
The loop completes on all five Stage 1 shapes, and you can point at the turn where it stopped asking.	Whether any answer is correct. Nothing here grades output.
The group of 12 escalates rather than resolving, which is the pass.	Tone. The abusive ticket comes back calm because nothing is watching.
Every tool call and every result is on the wire, and you read them.	Cost and speed. There is no bench yet, so any number is a guess.

Build 2 · Your own tools

Build 1 handed you nine tools already written. This one does not. You pick the tool, you write the description, you write the function, and you defend why that one and not the other two. No client hands you a tool list on day one either.

YOU WALK OUT WITH a tool you specified yourself, a trace where Claude reached for it unprompted, that same tool served over MCP, and what each of the two cost you in context.

The gap

Nine tools, and not one of them answers what a stranded customer asks first. When is the first day I can fly?

Three questions Larkspur customers ask every day, and nothing in the file to call for any of them.

Two of the three slots at the top of agent.py

Both have been on your screen, empty, since the first time you opened the file. This is the build that fills them.

WHERE	WHAT BELONGS IN IT
EXTRA_TOOLS	The schema. Name, input shape, and the description string Claude chooses from.
LOCAL_TOOLS	The function, registered under exactly that name. A schema with no function is a tool the loop cannot dispatch.

Three candidates

Every candidate is grounded in a document the client already owns. A tool whose answer you cannot point at invents flight facts, and that is gate one.

Pick one. Two or three is the ceiling, not the target.

CANDIDATE	WHERE ITS ANSWER COMES FROM
next_available_day	Live inventory in <code>data/americas/</code> . The function is imported at the top of <code>agent.py</code> and has been there all along. It was written and working, and nobody had told Claude it existed. Both slots are empty, so the schema is yours to write, name and input shape and description, and so is the line that registers it.

fare_rules

The Handbook excerpt in the repo: fare family, change fee, fare difference.

reopen_stats

The 600 transcripts: how often this shape came back inside 72 hours.

2.1 Your own tool, chosen unprompted

WHAT YOU ARE BUILDING

A tool the model never picks is a tool you never shipped.

Every schema you register goes out again on every turn, fired or not. One new capability is a permanent tax, and you are about to price it.

DO

1. Argue one tool off the shortlist before anyone types. Five minutes of talking buys you thirty of building.

```
$ python3 run.py --show-tools
```

2. Write the schema into `EXTRA_TOOLS`, then register the function in `LOCAL_TOOLS` under exactly that name. `next_available_day` is already imported at the top of the file. Anything else on the shortlist you write yourself. The usual break here is a schema with no function registered under its name.
3. Check it is registered before you spend a test message on it. Yours is listed beside the given nine, tagged `+ yours`, with the description you actually wrote.

```
$ python3 run.py --show-tools
```

4. Write your pod's probe. Three lines in `build2_probe.txt` at the repo root: PNR, last name, the customer's message. One question for the whole pod, so you test one thing instead of five. The message is a customer's sentence, not a tool call, and the gate reads what the model did with it.

```
K7PQ2M
Marchetti
When is the first day I can actually fly?
```

5. Write the probe before the description is finished. One reverse-engineered from a tool that already fires proves nothing.
6. Price it, before you argue about it. This counts every schema on the wire, tool by tool, and the per-tool column is what dropping that one tool would save. It is a property of the list, so it prices the same every time you run it.

```
$ python3 run.py --tool-tax
```

7. Run the five shapes again and put tokens in beside the two numbers you wrote down in Build 1. Up or down, both are real answers.

```
$ python3 run.py --all --trace
```

8. Then take the gate.

```
$ python3 verify.py 2.1
```

9. Back in the demo: ask when is the first day I can fly. Your own tool should appear in the tool line under the reply, on a message that never named it.

YOU WILL KNOW IT WORKED WHEN

The gate does not care that the tool exists. It cares that the model chose it, on a message that never named it. Three attempts, one success is enough, and declared-but-never-called does not pass. Read the last line as two different kinds of number. Tokens in is what one sampled conversation happened to cost, and it moves every run; the schema figure is what the list costs on every turn whatever the model does, and it does not.

```
$ python3 verify.py 2.1
```

Step 2.1: Build 2 • Your own tool

- ✓ at least one tool beyond the given nine (next_available_day)
- ✓ the new tool is registered in LOCAL_TOOLS
- ✓ next_available_day description is $\geq$ 40 characters (628)
- ✓ the model chose a new tool on a conversation that needs it (next_available_day, attempt 1 of 3)
- ✓ returned a non-empty string
 - (that probe cost 1,733 tokens in across 3 tool call(s), with another 9,168 read from cache and billed separately. The tenth tool is not free: the schemas alone are 2,600 tokens on every turn, counted on the wire. `python3 run.py --tool-tax` prints them one by one.)

All checks passed. Evidence code: 830-C95 (banked for "Alice Example")
That code is your receipt for step 2.1. Paste it into the build site to bank it. Nothing to upload.
<https://virtual.partnerbasecamp.com/build/>

IF NOT

YOU SEE

It never gets picked and you cannot see why

It only fires when you name it in the message

WHERE TO LOOK

If `--show-tools` lists yours tagged `+ yours`, it is registered and the suspect is the description. Go there first, and go there before you touch the probe.

Read your probe as the model sees it. Does that customer's message need your tool, or could one of the given nine answer it?

IN CLAUDE CODE

Type `claude` in the repo folder, then `/build`. Once your pod has argued one tool off the shortlist, before you type the schema.

I'm adding a tool to this agent. Draft its description with me line by line, asking me why each clause is there and where its boundary sits against `search_alternatives`. Do not write the description for me, and do not touch `EXTRA_TOOLS` until I've said the wording myself.

IF YOU ARE AHEAD · after the gate, never instead of it

The second tool. Add `fare_rules` or `reopen_stats`, run all five shapes again, and answer two questions off your own trace. With the given nine plus every one of yours going up on every call, did routing still hold on all five shapes, and what did tokens in per contact do? Past about fifteen tools routing starts to slip and the token tax climbs. End it with keep-it-or-delete-it.

The keep-or-delete line in `PITCH.md`, with the number that decided it.

NEXT

Step 2.2. Same tool, same schema, off your laptop.

2.2 The same tool, over MCP

WHAT YOU ARE BUILDING

Same schema, different transport. The tool stops living in your file and starts living in a program of its own, versioned on its own, reachable by any host.

The client asks the server what it has, Claude picks, the client calls it.

`tools/list`, then `tools/call`.

D0

1. Run the selftest and watch the two messages that go past. `tools/list` is the server handing over every tool it has, with its JSON schema. `tools/call` is one of them being run.

```
$ python3 support/mcp_selftest.py
```

2. `tool_list()` is the one function that decides what Claude is offered on every turn, and it is yours to change. The server's own tools come from `mcp_client.tools()`. Dispatch is handled already, so a call for a name the server owns reaches the server without you doing anything. Decide what goes into that list, and what has to leave `LOCAL_TOOLS` for the server to be the only owner of the name.
3. Check what Claude is offered. Yours should now be tagged `via mcp`.

```
$ python3 run.py --show-tools
```

4. Run your probe's question with the trace on and find `[mcp]` on the call. Same tool, same arguments, different owner.

```
$ python3 run.py K7PQ2M --trace --message "When is the first day I can actually fly?"
```

5. Now price the new list and put it beside your 2.1 number. Count the schemas rather than reading tokens in off one conversation. Tokens in moves with what the model chose to do. A schema count does not move at all.

```
$ python3 run.py --tool-tax
```

6. It moves by what the server offers, not by the transport. On the reference run the tax went from 2,600 to 2,895 tokens. The whole 295 is `fare_rules`, the second tool the server carries, and it arrives with the merge. Every schema you are offered rides on every turn, wherever it lives. MCP changes what you maintain, and does nothing at all for routing or for what the tool list costs you.
7. Then take the gate.

```
$ python3 verify.py 2.2
```

8. Back in the demo: ask that same question again. Same answer, same tool line, and nothing on the surface changed. The tool now lives in another program and the customer cannot tell.

YOU WILL KNOW IT WORKED WHEN

A tool discovered over MCP fired on your pod's probe, and the local copy is gone. One tool, one owner. The last two lines are the gate showing its work on the token question, and they are deliberately two different kinds of number: the schema count is the same every time you count it and it names the tool the move came from, while tokens in on the probe is one sampled conversation and will read differently for you.

```
$ python3 verify.py 2.2
```

Step 2.2: Build 2 · The same tool, over MCP

- ✓ the MCP server answers tools/list (fare_rules, next_available_day)
- ✓ at least one MCP tool is offered to Claude (fare_rules, next_available_day)
- ✓ no tool name is served twice, once here and once over MCP (clean)
- ✓ an MCP-discovered tool fired on the pod's probe (next_available_day, attempt 1 of 3)
- ✓ returned a non-empty string
 - (schema tokens on every turn: 2,600 at 2.1, 2,895 here, +295 for fare_rules. Tokens in moved by what the server offers (fare_rules), not by the transport. counted on the wire.)
 - (this probe cost 11,345 tokens in across 2 tool call(s). That figure moves run to run with what the model chose to do; the schema count above does not.)

All checks passed. Evidence code: 662-1D7 (banked for "Alice Example")
That code is your receipt for step 2.2. Paste it into the build site to bank it. Nothing to upload.
<https://virtual.partnerbasecamp.com/build/>

IF NOT

YOU SEE

WHERE TO LOOK

The tool shows up twice

The local copy is still registered. One name, one owner.

Nothing is tagged via mcp

Is the server process starting at all? Everything it wants to say to a human goes to stderr, so read stderr.

It fires, but tokens in went up

Count the schemas rather than arguing about the conversation: `python3 run.py --tool-tax`. How many tools went up on that turn, which of them are new, and is one of them the same tool twice?

IF YOU ARE AHEAD · after the gate, never instead of it

One server, many hosts. Register the same server in Claude Code, ask it when J5NU8S can fly, and watch a host you did not write drive your server. Then take it out again.

claude mcp add larkspur -- python3 support/mcp_server.py, then claude mcp remove larkspur when you are done.

NEXT

Pitch and ship. The build stops and the claim starts.

Provable now, and not yet

PROVABLE NOW	NOT YET
The model chose your tool unprompted, on a message that never named it.	Whether the tool's answer is any good. No case grades it.
The same tool, discovered over MCP, fired on the same probe.	That MCP bought you routing or cost. Counted on the wire, the schema tax moved by exactly the extra tool the server carries.
What the tool list costs on every turn, counted rather than sampled, before and after the merge.	What it costs per contact in dollars. That needs a bench, and the bench is next session.

Pitch and ship

The deliverable is a repo, not a recording. Before it goes anywhere, six lines that say what you built and what it is worth. One of those six names what is broken.

The pitch, six lines

"Compared to what?" is the question that separates a demo from a claim. Which of your six survives it?

LINE	WHAT IT CARRIES
Built:	What we built, and what you connected.
Does:	What it does for a stranded customer, in one line.
Number:	Before and after, off your own trace, with a unit and a denominator. The gate reads this line by name and wants all three: the figure, what it is measured in, and what it is per.
Guardrail:	The one you can prove, and how you proved it.
Next:	What you would do with another week.
Still broken:	The gap you name before anyone finds it. Scored like the other five. The gate reads this label by name.

Three things are provable at this point

Accuracy and dollars are not on that list. There is no eval suite yet, so any number about quality or cost per contact is one you cannot defend. Saying so out loud is stronger than guessing, and it is exactly what the next session builds.

Every build ends with a box like this one. Read the Build 4 version twice: it is the build where a number exists for the first time, and the first number is the easiest one to round up.

- Your Stage 1 pass rate.
- Your before and after turn count and context cost, measured by you.
- A forged confirmation token being refused, while a real customer click is accepted.

The template, then make every line yours

This is the shipped `PITCH.md`, byte for byte. Fill every line after the colon. `Lever:` stays as the menu until Build 4 picks one word; the gates read `Still broken:` and `Lever:` by name, so keep both labels exactly.

What each line wants. Built: what we built, one line. Does: what it does for a stranded customer. Number: before to after, with a unit and a denominator, measured on what. Guardrail: the one we can prove, and how. Next: what we would do with another week. Still broken: the gap we are naming before anyone finds it. One person types the pitch. Agree who: one hand per pod, because merge conflicts eat the whole thing.

The 4 lines under `## Priya asked` are not for today. They are the between-sessions job.

```
# PITCH.md
```

```
Six lines and a lever. Both of the last two are scored.
```

```
Built:
```

```
Does:
```

```
Number:
```

```
Guardrail:
```

```
Next:
```

```
Still broken:
```

```
Lever: <cost | speed | intelligence>
```

```
## Priya asked
```

```
Costs:
```

```
Wrong:
```

```
Runs it:
```

```
Left out:
```

Publish the pod's canon

One person pushes the canon at the end of the build. Agree who before the clock runs out. They tag last, once the canon is on the remote. The readout is generated from the agent being published, so it describes the version the pod is about to stand on.

```
$ python3 readout.py
$ python3 pod_sync.py --push-canon
$ git tag v1 && git push origin HEAD --tags
```

Then everyone else takes it, once that person has said it is up. Take a canon nobody has pushed yet and you pull down the last build's agent.

```
$ python3 pod_sync.py --take-canon
```

Post the repo URL

Post the repo URL in the pod thread. That is the list the overnight review runs from.

Prove the key never travelled

The first prints `.env`. The second must print nothing at all. Anything it prints is an incident rather than a tidy-up: stop, and say so out loud.

```
$ git check-ignore .env && git log --all --oneline -- .env
```

And this is what gets run on it

The rubric is public and it runs on the shared screen. Overclaiming scores worse than a gap.

Nothing is required between the sessions. The next session starts from the pod's canon, not from whatever your laptop still has.

DIMENSION	WHAT EARNS THE 2
Runs	The loop gate and the generalization gate both banked.
Generalizes	All five Stage 1 shapes, not the easy one.
Tools	Every description says when to call it and what comes back.
Guardrail	Tested, not judged. A forged token is refused.
Claim	A number you measured, and a claim sized to it.

Between sessions

One short job, and it is not a coding job. One file lands in your repo, written into it by a run the room lead makes, and the next session opens on it, so read it when it arrives. If it is not there, the repo was not shared: say so in the pod thread.

An agent read your build

It read your code, not your demo. It ran nothing, and it changed exactly one file: `FEEDBACK.md`, at the root of your own repo.

It is written into the pod repo before the next session opens, by a run the room lead makes across every pod that shared its repo, so it is not in a fresh clone and it is not something you generate. If `git pull` does not bring it down, the repo was not shared: say so in the pod thread rather than hunting for the file.

```
$ git pull
$ open FEEDBACK.md
```

What is in it

SECTION	WHAT IS IN IT
Priya's note	Her push, in her words: use your best model.
Still open	The same four questions, with what she means by each.
What the review agent found	Three to five observations, in your own file and function names.
Before our 9:00	Which model should we be on, and how will you prove it.

Her four questions

None of the four is answered by naming a model. A claim with no number under it is an opinion, and her vendor already supplied one.

SHE ASKED	WHAT SHE MEANS BY IT
What it costs	Per resolved contact, against the \$6.90 a human contact costs them.
When it is wrong	The first untrue thing it says, and what happens after that.
Who runs it	In June, after you have left.
What you left out	The scope you cut, and why.

Answer her four, before the next session opens

Before the next session, whoever is not touching `agent.py` answers Priya's four questions in `PITCH.md`, one line each. The file carries the 4 lines already, under `## Priya asked`. In your own words, and push them.

It is the one stretch of the two sessions where the person who does not write Python is the best-equipped member of the pod. Every one of her four questions is about the customer, the scope or the handover, and none of them is answered by reading the loop.

The review agent reads those 4 lines beside the code and says, for each one, whether the repo backs it. An answer with a number the bench files do not support comes back marked as not supported, which is exactly the read you want before you say it to a client.

```
Costs:
Wrong:
Runs it:
Left out:
```

THE LINE	WHAT IT ANSWERS
Costs:	Per resolved contact, against the \$6.90 a human contact costs them. Say model cost or loaded, and say which.
Wrong:	The first untrue thing it says, and what happens after that.
Runs it:	Who owns it in June, after your team has left.
Left out:	The scope you cut, and why.

First thing, next session

Before anything else, on every laptop. What your pod shipped is on the remote, tagged `v1`. What is on your laptop is a stale guess. Take it, run it once, and the pod is arguing about one agent again.

It saves your own `agent.py` into `.workshop/mine/` first and prints where it went. Nothing you wrote is lost.

```
$ python3 pod_sync.py --take-canon
$ python3 run.py K7PQ2M --trace
```

Build 3 · Build the proof

A claim is only as good as its instrument. You made a claim in the pitch and nothing has graded it. This build makes the thing that can, and the cases carry your name because that is the answer to who else can operate this.

YOU WALK OUT WITH an eval suite with a case authored by you, a surface a sponsor can watch, and a result you would read out loud even when it says **BLOCKED**.

The four pieces

PIECE	WHAT IT IS
The case	An input and what you expect back. Frozen, so it means the same thing later.
The run	Your agent, on that input, unmodified.
The grader	The thing that decides pass or fail, and the half everyone forgets is code too.
The gate	Whether a failure blocks a release on its own. Hard, or soft.

Which grader to reach for

Two habits before you ship a judge. Make it quote what it read, and never let a model grade its own output.

`eval_harness.py` holds itself to the second one. Your agent runs on the model `support/data.py` names. The judge runs on `claude-opus-4-8`, a more capable model, and up rather than down on purpose. A judge on the same model id as the agent shares the blind spot that wrote the answer, and reads its own phrasing back as correct. Up rather than down, because catching a fluent reply that contradicts the tool result it was handed is the one thing a judge is for. It costs more per case and it is slower, and that is this grader's charge, printed in the header of every run. Swapping it is one environment variable, and diffing the verdicts is the fastest way to see what your rubric actually rests on.

And suspect the eval first. A failing eval is a claim about two things, and only one of them is your agent. Larkspur's week 8 lost a suite to 37 of 48 in a single night, the only change was the rubric, and nine of eleven failures were the grader. Fixed in 40 minutes, agent untouched. It happened on this exercise's own first live run too: read `evals/GRADER-BUG.md` in your repo.

GRADER	GOOD FOR	CHARGES YOU

Exact match	A PNR, a flight number, a policy row id	Nothing, and it fails a right answer worded differently
Rule-based	Was <code>confirm_rebooking</code> called? Was a hotel promised? Roughly seven in ten production checks live here	You have to say the rule out loud
LLM as judge	Tone, acknowledgement, whether an explanation is honest	Money, latency, and a grader that can itself be wrong

3.1 Build the proof

WHAT YOU ARE BUILDING

Three surfaces, three audiences. The trace is for you, the chat is for the customer, the panel is for the sponsor. Most teams build the first, demo the second, and never build the third.

A blocked release can pass this one. Honest proof beats a green dashboard.

DO

1. The surface has been open since before the first build, and this is the step it was waiting for. Chat on the left, the evidence panel on the right, your own `run_agent` behind both. If the tab is closed, start it again. The surface is given so the build time goes into the proof, not the page.

```
$ python3 demo/serve.py
```

2. Copy the worked examples, then make them yours. A pod that ships them untouched does not clear the gate.

```
$ cp evals/cases.example.json evals/cases.json
```

3. Write one case per member, three minimum, each carrying its author's name in the `author` field. That field is matched against the name banking the gate. A pod of six cannot ship six cases with one person's judgment in all of them.
4. Make two of them hard gates. A hard gate is a case whose failure blocks a release on its own. If everything is soft, nothing is gated.
5. Run the suite against your own agent.

```
$ python3 eval_harness.py
```

6. Fill the panel. Pass rate, what blocked, one claim, the caveat. Behind on time, cut the panel: the cases are the proof, the panel only puts them on a screen.
7. Write the case a client would sign as the bar. Not the case that is easiest to grade, the one whose failure a client would call a failure. Put your own name in its `author` field. The gate matches that field against the name banking it, so this one is yours and nobody can bank it for you.
8. Then take the gate. It does not require your cases to pass.

```
$ python3 verify.py 3.1
```

9. Back in the demo: the evals panel on the right fills in, one row per case, with the hard gates marked and the release verdict under them. That panel is the third surface, and it is now reading your own cases.
10. Bring three things to the share-out. The case you wrote first and the customer shape behind it. How you graded it, and the sentence your expectation turns on. And the one your agent failed. Say that third one out loud: it is the most useful minute of the build.
11. One person pushes the canon at the end of the build. Agree who before the clock runs out. Everyone else takes it.

```
$ python3 readout.py
$ python3 pod_sync.py --push-canon
```

YOU WILL KNOW IT WORKED WHEN

Nine checks and one note, and every label names the file it read. Six read your cases and your last run: at least 3 cases, at least 2 hard gates, every case says what it expects, one authored by you, the last run was YOUR cases, the run covered at least 3 cases. Three read `PITCH.md`: the `Number:` line carries a figure and a unit and a denominator, the file is more than the shipped template, and the `Still broken:` line names one thing. The `Number:` check is the strict one, and it is strict on purpose. A figure with no unit and nothing to divide it by is not a claim anybody can check. Whether your cases passed is not on that list, and never will be. A blocked release passes this gate. Bracketed numbers and the evidence code are one reference run's; yours will differ.

```
$ python3 verify.py 3.1
```

Step 3.1: Build 3 · Build the proof

- ✓ at least 3 eval cases (5)
- ✓ at least 2 of them are hard gates (2)
- ✓ every case says what it expects
- ✓ at least one case is authored by you (Alice Example)
- ✓ the last eval_harness run was YOUR cases, not the examples
- ✓ the run covered at least 3 cases (ran 5 of the 5 in cases.json, 5 scored, release BLOCKED)
 - (the evidence panel has bench numbers to show.)
- ✓ `PITCH.md`: the 'Number:' line carries a figure, a unit and a denominator (15,968 to 13,135 tokens in per contact on K7PQ2M, ...)
- ✓ `PITCH.md`, whole file: more than the shipped template (251 words, floor is 40)
- ✓ `PITCH.md`: the 'Still broken:' line names one thing that still does not work

All checks passed. Evidence code: 8E5-8BB (banked for "Alice Example")
That code is your receipt for step 3.1. Paste it into the build site to bank it. Nothing to upload.
<https://virtual.partnerbasecamp.com/build/>

IF NOT YOU SEE

WHERE TO LOOK

Every case passes

You wrote them too gently. Ask what you would write if you were trying to find the thing that breaks, then write that one.

A case fails on behavior you believe was right

Suspect the rubric before the agent. Read `evals/GRADER-BUG.md`, which is that exact failure from Larkspur's own week 8.

The gate says the `Number:` line is incomplete

Read your own line back and ask what a stranger could check. It wants three things on that one line: a figure, what the figure is measured in, and what it is per. The hint names whichever of the three is missing.

A row reads `UNKN` and the run says the grader could not read its own judge

That case ran and was not scored, and it is not charged to your agent. The pass rate is over scored cases only. Run the suite again; if it repeats, the judge prompt for that case is the suspect, not the agent.

IN CLAUDE CODE

Type `claude` in the repo folder, then `/build`. Once you know which customer shape your case is about, before you fight the JSON.

Write the eval case I'm about to describe, then argue against it as the grader would: every way my expect could pass a bad answer or fail a good one. Ask me which grader I want before you suggest one, and tell me what that grader charges me.

IF YOU ARE AHEAD · after the gate, never instead of it

Agent, or grader? Take one failing case and decide, in writing, which one it was. If it was the grader, change only `expect`, bump `rubric_version` to `v3`, keep the v2 file beside it, and run it again. A verdict that flips with `agent.py` untouched is your own week 8.

Two rubric files, v2 and v3, side by side, and the verdict that flipped between them.

NEXT

Build 4. Now something in your repo can read a saving, you can go and make one.

Build 4 · Pull the lever

Proof came first for a reason. You cannot bank a saving you have no way to read. Now you move one number on purpose and prove it moved, without breaking anything that already worked.

YOU WALK OUT WITH one declared lever, a before and after pair off your own laptop, and a movement you would put in front of somebody who owns the budget.

What each lever buys, and what it charges

The lane is which number you are moving. The lever is the change you make to move it. The

Lever: line in `PITCH.md` names your lane, and the gate reads that one word.

"All three" is a dodge. Which one first, and why that one?

LEVER	BUYS	CHARGES
Intelligence	Harder shapes, fewer wrong answers	Tokens, latency, more to evaluate
Cost	A number the sponsor can defend	A cache you must keep valid, and a latency risk you have to measure rather than assume. The same lever read as a fifth slower at one run per shape and as no change at all at three
Speed	Abandonment, and the SLA	A smaller model. A quality bet.

Your lane, and where to look

Nothing new arrives in `agent.py` for this build either. The intelligence lane writes the third slot at the top, `TONE_ADDENDUM`, which has been empty since the first time you opened the file. The cost and speed lanes change `run_agent()`, the same function you fixed in Build 1.

Stage 2 is the harder set: five adversarial messages. A wire rule is a check on what actually went out, which is why the count only exists there. The intelligence lane is allowed to be a different model, one step up or one step down, as long as you bench the same stage on both sides of the swap. Its gate wants two things: one more Stage 2 case satisfying its rules, and a hard-gate suite that was failing now passing on more than half its runs. One row flipping once is a sampled tool choice, not a fix.

The speed lane is graded on p50, not on p95. At five or fifteen conversations p95 is literally the slowest single conversation in the run, and the slowest of fifteen is bigger than the slowest of five

for reasons that have nothing to do with your lever. `bench.py` prints it as slowest observed for that reason, and it still belongs in the pitch as the worst case you saw.

LANE	YOUR NUMBER	WHERE TO LOOK
<code>cost</code>	Model \$ per contact	Cache, tokens, what the tool list costs
<code>speed</code>	p50, with the slowest observed beside it	Triage, parallel reads, turns
<code>intelligence</code>	Wire rules passed on Stage 2	Five adversarial messages, one model either way

Real measurements, and not one of them a model upgrade

\$0.14 per resolved contact against \$6.90 for a human chat is the only comparison a board reads, and it flatters. Humans still take 42% of in-scope chats, and all voice. Say the caveat, and the date the prices came from, on the same slide.

Caching can also cost you more than it saves. Writing to the cache is priced above fresh input. If the front of the request changes on every call, you pay the premium and never collect. Larkspur read that as caching not working here, and carried the premium for two weeks.

`bench.py` reports model cost only. Larkspur's loaded cost per resolved contact was about \$0.14 against a model cost of \$0.087, so roughly 40% of the real number is infrastructure and evals. Report it as model cost, or say loaded and estimated. Do not quietly call it the loaded number.

CONFIGURATION	WEEK	P95	MODEL \$/CONTACT
One mid-tier model, no cache	2	9.8s	\$0.173
plus caching, parallel reads	5	6.9s	\$0.102
plus small-model triage, trimmed output	7	4.7s	\$0.087

A year of it, worked

Marcus asks one question about your number: per year, at our volume. It is one multiply, and doing it out loud is worth more than another decimal place on the number itself.

STEP	THE ARITHMETIC	COMES FROM
Disruption chats a year	about 530,000	The case's five lines
Humans still take 42% of in-scope chats	530,000 x 0.58 = about 307,000 the agent path can reach	Larkspur's week-12 containment

At \$0.14 a resolved contact	$307,000 \times \$0.14 = \text{about } \$43,000 \text{ a year}$	Larkspur's loaded number at week 7
The human baseline for those same contacts	$307,000 \times \$6.90 = \text{about } \$2,118,000 \text{ a year}$	The case's five lines
The gap	about \$2,075,000 a year	Subtraction, and it is not a saving yet

Say the caveat on the same slide as the number, in three parts. The 223,000 chats humans still take cost about \$1,540,000 a year and do not go away. All voice is out of scope, and voice is 59% of contacts. And a gap is not a saving until somebody does not buy something: Larkspur booked theirs as 60 seasonal BPO seats not bought, about \$410,000, because they ran at 50% exposure for one storm season rather than at 100% for a year.

Then do it again with your own after number instead of \$0.14, and say which one you used. Yours is model cost off `bench.py`; \$0.14 is Larkspur's loaded number. They are about 40% apart and a client will ask which.

4.1 Pull the lever, measure it

WHAT YOU ARE BUILDING

Bench before you tune. The before number cannot be rebuilt after.

A win that breaks a Stage 1 shape fails, and the gate checks both sides.

D0

1. Declare your lane in writing, in `PITCH.md` and in your pod thread. The shipped file carries the line as a menu, `Lever: <cost | speed | intelligence>`. Replace everything after the colon with the one word you picked, because the gate reads that word and refuses the menu.

```
Lever: cost
```

2. Write the caveat now, before anyone benches. One sentence naming what this lever will cost, in the client's words, and what the number about to arrive will not cover. Written after the number, it is a defense. Written now, it is a claim you can stand on.
3. Every member benches on their own laptop, labeled `before`, before any change lands. A podmate's fix does not carry your before number, and it cannot be rebuilt. Three runs per shape, both sides, and the gate requires it. At one run per shape an output-token delta under about 15% is noise. `--compare` would print it beside your win as if it were real. `--cold` makes the run pay for its own cache instead of reading one an earlier run already paid for.

```
$ python3 bench.py --label before --stage 1 --runs 3 --cold
```

4. Intelligence lane only: bench Stage 2 on both sides of the swap as well. Your metric is the wire-rule count, and that only exists on Stage 2.

```
$ python3 bench.py --label s2-before --stage 2 --runs 3
# then, after the swap
$ python3 bench.py --label s2-after --stage 2 --runs 3
```

5. Pull one lever. The one you declared.
6. Bench again and diff the two labels. Same stage and the same number of runs on both sides, or the comparison is not one. Then read the cache line under the table. A run that read a cache it never wrote is priced on a discount an idle agent in production never gets. `bench.py` says so out loud when it happens.

```
$ python3 bench.py --label after --stage 1 --runs 3 --cold
$ python3 bench.py --compare before after
```

7. Walk the five shapes again before you call it done. A win you did not recheck is a claim you cannot make.

```
$ python3 run.py --all --trace
```

8. Then take the gate.

```
$ python3 verify.py 4.1
```

9. Back in the demo: the number at the top of the panel fills in, with what it is per and what it is measured against, and the before and after rows fill in under it. That is the sponsor's view of the lever you pulled.
10. One person pushes the canon at the end of the build. The last hand. Agree who before the clock runs out. Everyone else takes it.

```
$ python3 readout.py
$ python3 pod_sync.py --push-canon
```

YOU WILL KNOW IT WORKED WHEN

Your declared lane's number moved at least 10% in the right direction, and Stage 1 still resolves 5/5. Ten percent is there so noise does not read as a win. A 70% cost win that breaks a shape fails. The comparable check is the one people trip over: it wants the same stage, the same model, and at least three runs per shape on both sides, because one run per shape is a sample and the floor it would be clearing is smaller than the noise. The first check reads the `Lever:` line in `PITCH.md`. The numbers below are one reference run on the cost lane; yours will differ.

```
$ python3 verify.py 4.1
```

Step 4.1: Build 4 · Pull the lever

- ✓ `PITCH.md`: the 'Lever:' line names one lane (cost)
- ✓ a 'before' bench exists
- ✓ an 'after' bench exists
- ✓ the two runs are comparable (3 runs per shape each)
- ✓ stage 1 still resolves 5/5 after your change (100%)
- ✓ cost lane: `model_cost_per_contact` 0.0564 → 0.0233 (-59%)

All checks passed. Evidence code: 61A-096 (banked for "Alice Example")
That code is your receipt for step 4.1. Paste it into the build site to bank it. Nothing to upload.
<https://virtual.partnerbasecamp.com/build/>

IF NOT
YOU SEE

WHERE TO LOOK

The number went the wrong way

Read the bench file before you touch the agent again. Cache read and written, tokens, turns. The row that moved is not always the row you aimed at.

The bench says `cache: not used on any turn`

Nothing has been marked for reuse, so a stable prefix has nothing to hit. That is not a 0% hit rate; it is no cache at all. What would the API have to be told, about the part of the request that does not change, before it can keep anything between calls?

Caching reports a 0% hit rate

Something is being kept and nothing is ever found again. Print what goes out at the very top of every request, two calls in a row, and ask which characters are not identical.

Cost fell and latency rose

Check how many runs per shape are behind both numbers before you write that down. The reference lever read a fifth slower at one run each and no change at three. If it holds at three runs, it is a real charge and it belongs in the pitch: a lever that only shows its buys is a claim the client will find the charge on.

The bench says the run read a cache it did not pay to write

Somebody already paid for that cache, minutes ago, and this run read it for free. Production re-pays that write after every idle gap. Re-bench with `--cold`, or put the word warm next to the number in the pitch.

The gate says the two runs are not comparable

Read the stage, the runs per shape, and the model it prints for each side. One run per shape is a sample, not a measurement, and the gate will not compare below three.

IN CLAUDE CODE

Type `claude` in the repo folder, then `/build`. After the before bench, while you are deciding which lever to pull.

Read `.workshop/bench-before.json` and my latest trace, and tell me where the tokens go per turn. Ask me which lever I think that points to, and why, before you say yours. Do not change `agent.py`.

IF YOU ARE AHEAD · after the gate, never instead of it

The second lever. Leave `before` and `after` untouched, they are your banked pair. Pull a second lever, bench it as `after2`, and compare `after` against `after2`. The payoff is not the row that improved. It is the row that got worse, with its percentage. That is the Charges column, on your own build.

The row that got worse, named with its number, in `PITCH.md`.

NEXT

Pitch the client. Somebody who owns a budget is about to ask you what a year of this costs.

Provable now, and not yet

This build is where a number exists for the first time, so it is where the temptation to round up lives. A pod that names the loaded-cost gap before a client finds it is the pod that gets asked back.

PROVABLE NOW	NOT YET
Your declared lane's number moved, before and after, on your own laptop, 3 runs per shape.	The loaded number. <code>bench.py</code> reports model cost, and Larkspur's loaded figure ran about 40% above it.
Stage 1 still resolves 5/5 after the change, so the win did not break a shape.	Accuracy at volume, and anything on real traffic.
What the lever charged you, if you benched 3 runs per shape on both sides. The reference cost lever read a fifth slower at 1 run each and no slower at 3, which is the noise floor on the pack's own headline number.	That the same lever holds on a storm day, at concurrency, against a slow backend.

Pitch the client

No neutral audience. Whoever is not pitching is the client, and clients own budgets. You are paired across lanes, so the client opposite you moved a different number and knows exactly where your claim gets thin.

How the twenty five minutes run

PHASE	MIN	WHAT HAPPENS
You pitch	5	Demo the surface, state the claim, name the caveat
They are the client	4	Two questions, off the four axes. Not four soft ones.
Swap, they pitch	5	Same three beats
You are the client	4	Two questions back
Everyone writes	3	Your three lines, filed as an issue on the repo you just heard
Regroup	4	Back in the main room once your issue is filed

Three things go in with you

Claim only what your evidence shows. "We did not measure that" beats a guess, and the caveat is scored, so a pod that hides a gap loses more than a pod that owns one.

- One claim. A number, its unit and its denominator, on the one metric your pod declared.
- Two numbers. Before and after, straight off your own two bench files.
- One caveat. What you did not measure, named by you before she finds it.

Pick a persona, ask two questions

If your number is cost or speed, rehearse with Marcus. He is the harder room. He asks what a year of this costs and whose name is on it.

Your questions are graded too. Four soft ones and the room learns nothing. Ask them the question you would least like asked of your own build.

THE AXIS	PRIYA • PROJECT MANAGER	MARCUS • HEAD OF OPS
cost	Per resolved contact	Per year, at our volume
wrong	The first untrue thing it says	Who is accountable when it is

<code>operate</code>	Who runs it in June	The day your team leaves
<code>scope</code>	What you left out, and why	Where the kill switch lives

And one more chair, the one that can stop it

Priya and Marcus both want this to work. The third person in the room does not, yet, and they can end it in one meeting. Ask all 3 of these when you are the client, and have all 3 answered when you are not.

THE QUESTION	WHERE YOUR ANSWER COMES FROM
Whose data does it touch?	Appendix D's reads table. <code>lookup_booking</code> refuses a name that is not on the booking, <code>search_alternatives</code> takes the PNR alone, and the one read that is not fenced to the customer's own booking is <code>get_flight_status</code> . Name that one yourself.
What happens after the first untrue answer?	Which gate catches it, what the customer sees next, and where the case goes. <code>escalate_to_human</code> files a written summary, and <code>send_confirmation</code> is the only write with no threshold, so it carries whatever you got wrong.
Who signs a policy change?	The policy table has a named owner and a version. A rule change is a row edit with a changelog line, not a prompt release, and that is the answer that makes this operable in June.

Three lines, filed where the work lives

A number you would take to your own client, a claim you would not sign, and a gap. Filed as a signed issue on their repo, where the build lives, rather than in a chat that closes when the session does.

Then read the one on yours before the close. Every pod leaves with a read on their work that nobody running the room wrote.

What is provable now

Be precise about the claim. That precision is the last thing this build teaches, and it is the half that travels.

Provable, and not yet

That is the honest state of most week-four engagements.

PROVABLE	NOT YET
Three gated eval cases, authored and run	Accuracy at volume
Your own before and after on a metric you chose	Real customer traffic
A forged confirmation token refused	Anything at production scale

Two sessions, one agent

SESSION ONE • DONE	SESSION TWO • DONE	FROM HERE • YOURS
Build. The harness: the loop, the tools, and one tool you wrote yourself.	Prove. Eval cases, a grader, and a number that moved.	Scale. Nobody in this room does that part for you.

Write it down now

`ACCOUNT.md` is in your repo, at the root, with 3 fields and nothing else in it. Fill them before you close the laptop, while the build is still in your hands, and commit it with the readout.

Account:
Workflow:
Date:

THE LINE	WHAT COUNTS
Account:	A logo. Not a segment, not a vertical, not "a large insurer".
Workflow:	One job you understand end to end, the way you now understand a disruption chat: who starts it, what it reads, what it may change, and what stays human.
Date:	A date inside 30 days, on a calendar, for a conversation that is in somebody's diary or is about to be.

Then run the client half of the readout and take that with you. Same evidence, one page, in the words the person who signs uses: what it does, what it is fenced from doing, what is proved and by which check, what is not measured yet, your `Still broken:` line, and the 3 lines you just wrote.

```
$ python3 readout.py --client
```

The proof pack, before you go

The named account is the ask. A repo nobody takes to a customer inside 30 days is a training exercise. Not a segment, not a vertical: a logo, a workflow you understand end to end, and a date on a calendar.

You keep the repo, the eval cases and the grader. The grader is the half that travels, and it is the cheapest rehearsal available before a real pitch.

WHAT YOU RECORD	WHERE IT COMES FROM
The repo URL	Tagged <code>v1</code> when you shipped it.
The six <code>PITCH.md</code> lines	Written at the pitch, sharpened when you pitched the client.
Your two bench numbers	<code>bench.py</code> , before and after.
One named account, its use case, and a date inside 30 days	<code>ACCOUNT.md</code> , 3 lines, committed. You. Nobody else can write this line.
One page in the client's language	<code>readout-client.html</code> , from <code>python3 readout.py --client</code> . What it does, what stops it, what is proved.
What this agent cannot do, and what stops it	Appendix D, and the same table inside <code>readout-client.html</code> . Hand this one to a CIO.
Anything you took past a gate	The stretch box on your build steps. Not scored.

A · Working in one repo

What is in the repo

WHERE	WHAT IT IS
<code>guide/</code>	This document. Open <code>index.html</code> , or read the PDF on paper.
<code>setup.py</code>	Checks this machine. Run it until the last line says READY.
<code>agent.py</code>	The file you edit. About 200 lines: the loop and three empty slots on top, and below the fold, what Claude is told about each of the nine tools.
<code>run.py</code>	Runs the agent on one ticket and shows every turn on the wire. The command you type most.
<code>verify.py</code>	The gates. Run by step id.
<code>support/</code>	The nine tools' real functions. Given, correct, and never edited.
<code>bench.py</code> · <code>readout.py</code>	Measure the agent, and write the page that shows what it did.
<code>eval_harness.py</code>	Runs your pod's own eval cases against your agent.
<code>demo/</code>	The sponsor surface: chat on the left, evidence on the right. Given, and it runs as shipped.
<code>data/americas/</code>	Bookings, flights, and the policy engine.
<code>pod_sync.py</code>	The one <code>agent.py</code> your pod agreed on: push it, take it, or <code>--status</code> .
<code>evals/</code>	Worked examples now. Your own cases later.
<code>PITCH.md</code>	Your pod's six lines, and the 4 answers you write between the sessions.
<code>ACCOUNT.md</code>	The account you take this to, its workflow, and a date inside 30 days. 3 lines, filled at the close.
<code>TEAM.md</code>	Your pod's name and roster, typed once by whoever created the repo. The review reads the names.

`build2_probe.txt`

Your pod's one test message for a tool you write yourself. You create this one.

`readout.html` · `readout-trace.json`

Written at the repo root by `readout.py`, and committed when the canon is published.

`readout-client.html`

The same evidence in the client's words, from `python3 readout.py --client`. One page you can send.

The whole path

Five commands carry the build, and the repo's README carries the same five. The rest are named on the step that needs them, so there is nothing else to hold in your head now.

In this repo, `claude` gives you `/setup`, `/build`, `/check` and `/readout`. `/build` builds the step with you and never hands you the answer, and saying build the agent in plain words does the same, and `/check` runs the gate and translates what it printed, which is the shortest way in if you would rather not lead with a terminal.

Two more appear at the close, and neither is on the clock: `python3 readout.py --client` writes the page you can send a client, and `ACCOUNT.md` is 3 lines you type by hand.

```
$ git clone <your pod repo URL>      # once
$ python3 setup.py                   # until it says READY
$ python3 run.py K7PQ2M --trace       # every build
$ python3 verify.py 1.2               # every gate
$ python3 pod_sync.py --take-canon    # at the end of a build
```

The rule, in one sentence. **Nobody commits `agent.py` mid-build. At the end, the build's committer publishes the canon and everyone else takes it.**

```
$ python3 pod_sync.py --push-canon    # the committer
$ python3 pod_sync.py --take-canon    # everyone else
```

Everyone builds their own `agent.py` on their own laptop. Several people editing one file in one repo at the same time produces a merge conflict inside a loop they are all still learning to read, and no build has time for that. So the repo only ever holds one version: the one the pod chose.

`--take-canon` saves your version into `.workshop/mine/` first and prints the path. Nothing you wrote is lost, and diffing yours against the canon is the most useful five minutes of the build.

`PITCH.md` and `evals/cases.json` are the pod's shared record and they are normal commits. The bench pair never leaves your laptop.

Who publishes the canon

One person pushes the canon at the end of the build. Agree who before the clock runs out. Everyone else takes it.

Two canon pushes in one build is the merge conflict this whole protocol exists to prevent, so it has to be one hand and the pod has to have said whose before the clock is out.

Behind, or something broke

The pod repo is the checkpoint. This restores what the pod pushed.

```
$ python3 pod_sync.py --take-canon --force
```

What it does not restore

If `.workshop/` is already gone, say so rather than re-benching `before` against the current agent. That number would be a fiction, and your pod's claim rests on it.

NOT RESTORED	WHAT TO DO
<code>.env</code> , your key	Yours, and never in the repo. Re-create it from <code>.env.example</code> .
<code>.venv/</code>	Rebuild it with <code>python3 setup.py --fix</code> .
<code>.workshop/</code> , your banked gates and bench numbers	Not on the remote. Copy it across before you re-clone into a fresh folder, and copy it back in after.

B · The five tickets

One example of the bar, not the only wording that passes. Two of the five are not what they look like: one should not resolve at all, and one gets an answer this build has no gate for.

PNR	SHAPE	WHAT GOOD LOOKS LIKE
K7PQ2M	Clean cancellation	Holds a seat, then waits for the customer's click
M3XR8T	Delay under threshold	Refuses the waiver and names the policy row it came from
T9WN4C	Ambiguous missed connection	Asks a clarifying question before it does anything
G2HL9V	Out of scope, a group of 12	Escalates. It does not resolve, and that is the pass
R8KD3F	Abuse plus a legal threat	Comes back calm and helpful. There is no tone gate yet

C · The words of the build

WORD	WHAT IT MEANS HERE
PNR	The booking reference, six characters. It is how every tool finds this customer.
The wire	The requests and replies that actually went to Claude and came back. <code>--trace</code> prints it, and every gate reads it.
Turn	One request plus one reply. Nothing carries over on its own: you send the whole transcript again, and that is all the context Claude has.
<code>stop_reason</code>	The field that ends a turn or continues it. <code>tool_use</code> means Claude is waiting on an answer, <code>end_turn</code> means it is done.
<code>tool_result</code>	The block you send back with what a tool returned, keyed to the id of the request that asked for it.
Slot	One of the three empty lines at the top of <code>agent.py</code> , each labelled with the build and step that fills it. Empty from the first minute, so nothing new ever arrives in the file.
The fold	The ruled line partway down <code>agent.py</code> . Above it, the loop and the slots. Below it, what Claude is told about each of the nine tools.
Schema	The name, the input fields and the description Claude reads for one tool. Not the function that answers it.
Tool description	The only thing Claude reads to pick a tool. A briefing, not docs.
Shape	One kind of customer situation. The five tickets are five shapes.
Stage 1	The five tickets in the case. Every build has to still resolve all five.
Stage 2	The harder set: five adversarial messages. Only the intelligence lane benches it.
Gate	A check your agent passes before the build moves on. <code>python3 verify.py <step></code> runs one.
Hard gate	One failure blocks the release. Never an average.
Evidence code	What a passed gate prints on its last line. Your receipt, made from your name, one per person. Paste it into the build site.

Canon	The one <code>agent.py</code> the pod agreed on, published at the end of a build. Until you take it, your laptop is a guess.
Probe	Your pod's one test message for a tool you wrote, three lines in <code>build2_probe.txt</code> : PNR, last name, the customer's message.
Readout	The one page your repo writes about what your agent is and what it just did, from <code>python3 readout.py</code> .
Bench	One timed run of the agent over a fixed set of tickets, labeled so two runs can be diffed. <code>bench.py</code> calls one of these a sweep.
Lane	Which number you are moving in Build 4: cost, speed or intelligence. The <code>Lever:</code> line in <code>PITCH.md</code> names it.
Lever	The one change you make to move that number.
Prefix	The front of a request, the part that does not change between calls. It is the part caching can reuse.
p50	The response time half of contacts come in under. The middle one, and the number the speed lane is graded on.
p95	The response time 95 of every 100 contacts come in under. It is the number an SLA is written against, not the average. At the sample sizes a build reaches it is literally the slowest conversation in the run, which is why <code>bench.py</code> prints it as slowest observed and the speed lane is graded on p50.

D · What this agent cannot do, and what stops it

This is the page to hand a CIO. It answers the two things they ask first, whose data it touches and what stops it doing something nobody authorized, and it answers them from the code rather than from a slide.

Every line here is sourced from `support/tools.py` and `data/americas/disruption_policy.json`. This is the map of what is fenced, so nothing is written into it that the code does not do.

`python3 readout.py --client` puts these same two tables on a one-page HTML file you can send. Same wording, no retyping.

Reads

TOOL	WHAT IT DOES
<code>lookup_booking</code>	Takes a PNR and a last name, and refuses if the name is not on that booking. Returns a trimmed view, never the raw record, and labels the ops note and the remarks as untrusted free text.
<code>get_flight_status</code>	Takes a flight number and a date. The one read not fenced to the customer's own booking, so it is the one that can be pointed anywhere.
<code>search_alternatives</code>	Takes the PNR alone, so it can never be pointed at a route the customer did not buy.
<code>check_policy</code>	Re-derives fare family, loyalty tier and whether this is an overnight from the booking on every call, so a model cannot talk its way into an entitlement the data does not support.

Writes

TOOL	WHAT IT DOES	THE FENCE
<code>hold_seat</code>	Puts a seat aside and hands back a hold that expires in 15 minutes.	Reversible, and it undoes itself if nobody confirms.
<code>issue_voucher</code>	Issues on its own only under the policy threshold: 25 dollars for a meal, 40 for ground, 75 for goodwill.	A hotel is never automatic, and anything over the line queues for a human.

<code>confirm_rebooking</code>	Reissues the ticket. The irreversible one.	Only with the customer's own click token. "The customer said yes" in chat is not it.
<code>send_confirmation</code>	Writes the message the customer actually reads.	The only write with no threshold and no token, so it carries whatever the agent got wrong.
<code>escalate_to_human</code>	Files the case with a written summary for whoever picks it up.	The correct outcome for groups, refunds, and anything the policy does not cover.

BASECAMP

Prepared for Partner Basecamp participants. Not for reproduction or redistribution as training material. You are free to apply these patterns in your own client work. Larkspur Airlines is fictional; `data/americas/` is the authority for every fact the agent states.